

Chapter : 1

Introduction

1.0	Objectives
1.1	Introduction
1.2	Java Buzzwords
1.3	Object Oriented Programming
1.3.1	OOP principles
1.4	Summary
1.5	Check your Progress - Answers
1.6	Questions for Self - Study
1.7	Suggested Readings

1.0 OBJECTIVES

Dear Students,

After studying this chapter you will be able to :

- discuss the background of development of Java
- explain with the Java buzzwords summed up by the Java committee.
- discuss what is meant by Object Oriented Programming.
- state the principles of Object Oriented Programming.

1.1 INTRODUCTION

Java was conceived by James Gosling, Patrick Noughton, Christ Warth, Ed Frank and Mike Sheridan at Sun Microsystems Inc. in 1991. The language was initially called **Oak** but later renamed as **Java** in 1995.

The primary need for the development of Java was the necessity of a platform independent language which could be used to create software for embedded systems. We know that languages like C and C++ can run on a variety of CPUs, however a compiler needs to be developed for every CPU on which the program is to be run. Compilers are expensive and time consuming to create. Therefore, during the development of Java the main emphasis was on creating a platform independent language which could run on a variety of CPUs under various environments. This led to the development of Java.

At around the same time the World Wide Web emerged and Java became a language of the Internet. The Internet consists of various types of computers operating under different operating systems and CPUs. It is necessary to run the same program on all these environments and Java had the capability to run on various platforms and thus offer portable programs. The Internet ultimately led to the phenomenal success of Java. Java has an immense effect on the Internet. In a network, two broad categories of objects are transmitted between the server

and the client. The first category is the passive information and the second one is dynamic, self executing programs. Such a dynamic program is generated by the server, however it is active on the client computer. As such, these network programs pose serious difficulties in terms of security and portability.

We can create two types of programs in Java, **applications** and **applets**. An **application** is a program which runs on your computer under the operating system of your computer. An **applet** is an application which is developed to be transmitted over the Internet and executed by a Java compatible web browser. An **applet** is an intelligent program, which means it can react to the user input.

We know that everytime we try to download programs, we risk the possibility of infecting our systems with virus. Also other programs may exist which may try to gather private information such as credit card numbers, bank account numbers, passwords etc. by searching our computer files. Java provides a security against both these attacks by providing a **firewall** between the network application and the user's personal computer. This ability to maintain the security of the user's computer is considered to be one of the most important aspects of Java.

As we have already seen Java also satisfies the need of generating portable executable code. Both these aspects of security and portability are addressed by Java because the output generated by Java compiler is not executable code, but **bytecode**.

Bytecode is a set of instructions which are designed to be executed by the Java run time system called the **Java Virtual Machine**. Thus the **JVM** is the interpreter for the bytecode. Only the JVM needs to be implemented for each platform on which the Java programs are to be run. Although the details of JVM differ from machine to machine all of them interpret the same bytecode. This helps in creating portable programs which can run on a variety of CPUs. The execution of a Java program is under the control of the JVM, therefore it makes the program secure and ensures that no side effects outside the system are generated. Safety is also enhanced by the features provided in the language itself. The **bytecode** also makes the Java run time system execute programs faster.

1.1 Check Your Progress.

1. Fill in the blanks.

- a) _____ is a set of instructions designed to be executed by the Java run time system.
- b) The _____ is the interpreter for bytecode.
- c) Java is a _____ independant language.
- d) An applet is a dynamic program which is active on the _____ computer.

1.2 JAVA BUZZWORDS

This section describes briefly the list of buzzwords as summed up by the Java team. From among the buzzwords two viz. security and portability have already been addressed in the previous section. Here we shall see the remaining ones.

Simple : Java was designed to be easy to learn and effective to use. With prior programming experience, it is not difficult to master Java. Java provides a small number of clearly defined ways to accomplish a given task. Java inherits the syntax of C and C++ and many of the object oriented features of C++. Therefore users who have already learnt these languages find it easy to master Java with very little effort.

Object Oriented : Java is not designed to be source code compatible with any other language. The object model in Java is simple and easy to extend. The simple types, like integers are kept as high performance non objects.

Robust : The ability to create robust programs was given a very high priority during the design of Java. Java is a strictly typed language and restricts the programmer in a few key areas. It checks the program code at compile time as well as at run time. Java manages memory allocation and deallocation on its own unlike in C/C++ where the programmer has to manually allocate and free all dynamic memory. In Java, deallocation is completely automatic. Java provides object oriented exception handling.

Multithreaded : Java was designed to meet the real world requirement of creating interactive networked programs. Therefore to accomplish this, Java supports multithreading. Multithreaded programming allows you to write programs that can do many things simultaneously.

Architecture Neutral : Upgradation of operating systems, processor upgrades, and changes in the core system resources can all be the factors which may cause a program which was running successfully to malfunction at a later point of time. Java and the JVM has been designed in such a way so as to attempt to overcome this situation, which would enable the programs to run inspite of severe environmental changes.

Interpreted : We know now that the output of the Java compiler is the **bytecode** which is platform independent. This code can be interpreted by any system which provides a **Java Virtual Machine**.

Distributed : Java handles TCP/IP protocols. Accessing a resource using URL is similar to accessing a file. Java has a package called **Remote Method Invocation (RMI)** which brings a very high level of abstraction to client/server programming.

Dynamic : Java programs have the capability to carry a sufficient amount of run time type information. This information can be used to verify and resolve accesses to the objects at run time. This makes it possible to dynamically link code in a safe manner.

1.2 Check Your Progress.

1. Write True or False.

- a) Java is a strictly typed language.
- b) Multithreaded programming allows to write programs that do many things simultaneously.
- c) Java does not check program code at run time.

1.3 OBJECT ORIENTED PROGRAMMING

Object oriented programming is at the core of Java. All Java programs are object oriented. Let us therefore study the Object Oriented Programming Principles before commencing writing programs in Java.

We have already studied that all computer programs consists of two elements : code and data. A program can be organised either around its code or around its data. When a program is organised around the code we call it as a **process oriented** model where code acts on data. The process oriented model has been successfully implemented in procedural languages like C. However, as the programs grow in size they become more complex. In the second approach which is called **object oriented** programming, a program is organised around its data i.e **objects** and a set of interfaces to that data. Thus an object oriented program is characterized as data controlling the access to the code.

An important element of object oriented programming is **abstraction**. Abstraction allows you to use a complex system without being overwhelmed by its complexity. A powerful way of managing abstraction is through the use of hierarchical classification. The data from a traditional process oriented program is transformed into its component objects through abstraction. A sequence of process steps then becomes a collection of messages between these objects. Each object defines its own unique behaviour. These objects are treated as concrete entities. You tell them to do something by sending messages. This forms the basis of object oriented programming.

1.3.1 OOP Principles :

Object oriented programming languages provide a mechanism which helps to implement the object oriented model. These are given herewith :

Encapsulation : This is the mechanism which binds the code and the data that it manipulates. It thus keeps both the code and the data safe from outside interference and misuse. Access to the code and data is possible only through well defined interfaces. With encapsulation everyone knows how to use the code regardless of the details of the implementation. The basis of encapsulation in Java is the **class**. The class defines the structure and behaviour i.e. data and code that will be shared by a set of objects. Each object of a given class contains the structure and behaviour defined by the class. The objects are referred to as an **instance** of a class. Thus a class is a logical construct and an object is a physical reality.

The code and the data that constitute a class are collectively called as **members** of the class. The data defined by the class are referred to as **member variables** or **instance variables**. The code which operates on the data are referred to as **member methods** or **methods**. The methods define how the member variables are used.

Inheritance : This is the process by which an object acquires the properties of another object. Inheritance supports the concept of **hierarchical classification**. With the use of inheritance, an object would need to define only those qualities that make it unique within its class. It can inherit the general attributes from its parent class. eg. if you wanted to describe animals in an abstract manner you would say they have the attributes like size, intelligence etc. They also have certain behavioural pattern like they eat, breathe, sleep. Such attributes and behaviour makes up the class definition of animals. A more specific class of animals could be mammals with some specific attributes like mammary glands. This class would be a subclass of animals and the animals class is the superclass. Mammals would inherit all the properties of the animal class and have its own specific attributes also. You can have more than one level of inheritance where a deeply inherited subclass will inherit all the attributes from each of its ancestors in the class hierarchy. If a given class encapsulates some attributes then any subclass will have the same attributes i.e it will inherit all the attributes of all its ancestors, as well as add its own specific characteristics.

Polymorphism : This is a feature which allows one interface to be used for a general class of actions. Polymorphism is expressed as “one interface, multiple methods”. Thus it is possible to design a generic interface to a group of related activities. Polymorphism helps in reducing complexity by allowing the same interface to be used to specify a general class of action. The compiler would select the specific method as it applies to each situation. The programmer needs to know and use only the general interface.

Thus through the application of object oriented principles, the various parts of a complex program can be brought together to form a cohesive, robust and maintainable whole. Every Java program involves encapsulation, inheritance and polymorphism. This means that the student should bear in mind that every Java program is object oriented and study how this is applied in the chapters that follow.

1.3 Check Your Progress.

1. Match the following.

Column A	Column B
a) Polymorphism	(i) is a logical construct
b) class	(ii) One interface, multiple methods
c) Inheritance	(iii) is a physical reality
d) object	(iv) supports the concept of hierarchical classification

1.4 SUMMARY

Java was conceived by James Gosling, Patrick Noughton, Christ Warth, Ed Frank and Mike Sheridan at Sun Microsystems Inc. in 1991. The language was initially called Oak but later renamed as Java in 1995.

Java provides security by a firewall between the network application and the user's personal computer. This ability to maintain the security of the user's computer is considered to be one of the most important aspects of Java.

The output generated by Java compiler is not executable code, but bytecode. Bytecode is a set of instructions which are designed to be executed by the Java run time system called the Java Virtual Machine. Thus the JVM is the interpreter for the bytecode. Although the details of JVM differ from machine to machine all of them interpret the same bytecode.

Following is the list of buzzwords as summed up by the Java team.

Simple, Object Oriented, Robust, Multithreaded, Architecture Neutral, Interpreted, Distributed, Dynamic, Object oriented, OOP Principles, Encapsulation, Inheritance, Polymorphism

Source : *blog.ibe solutions.com*

1.5 CHECK YOUR PROGRESS - ANSWERS

- 1.1 1.** a) Bytecode
b) Java Virtual Machine
c) platform
d) client
- 1.2 1.** a) True
b) True
c) False
- 1.3 1.** a) - (ii)
b) - (i)
c) - (iv)
d) - (iii)

1.6 QUESTIONS FOR SELF - STUDY

1. Describe the evolution of the Java Programming language.
2. Write a short note on the Java Buzzwords
3. What is byte code? What do you understand by JVM?
4. Explain Object oriented programming. Describe in brief the OOP principles.

1.7 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net



This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Chapter : 2

Beginning With Java

2.0	Objectives
2.1	Introduction
2.2	Comments, Whitespaces, Separators
2.2.1	Comments
2.2.2	Whitespaces
2.2.3	Separators
2.3	Identifiers, Literals and Keywords
2.3.1	Identifiers
2.3.2	Literals
2.3.3	Keywords
2.4	Data Types
2.4.1	Integers
2.4.2	Floating types
2.4.3	Characters
2.4.4	Boolean
2.5	Variables
2.6	Type Conversion and casting
2.7	Arrays
2.8	Operators
2.8.1	Arithmetic Operators
2.8.2	Bitwise Operators
2.8.3	Relational Operators
2.8.4	Boolean Logical Operators
2.8.5	The Assignment Operators
2.8.6	The ? : Operator
2.8.7	Operator Precedence
2.9	Summary
2.10	Check your Progress - Answers
2.11	Questions for Self - Study
2.12	Suggested Readings

2.0 OBJECTIVES

Dear Students,

After studying this chapter you will be able to :

- + Discuss the keywords, identifiers and literals of the Java language
- + Explain how to write simple programs in Java and compile them
- + Describe the various data types in Java
- + Discuss variables and scope rules of variables
- + Explain and understand how arrays are implemented in Java
- + State the different operators in Java and operator precedences

2.1 INTRODUCTION

We begin our study of Java by understanding the basics like **keywords**, **identifiers** and **literals**. We also study the simple data types viz. **integers**, **floats** and **characters** and the different types of operators supported by Java in this chapter.

Let us begin our study of Java by writing a small program to understand how to compile and run a Java program.

```
/* A first sample program
this is the way to give multiline comments */
class First {
    //Another way of writing comments in the program
    public static void main(String args[])
    {
        System.out.println("My first program in Java");
    }
}
```

In Java, the source file is called a **compilation unit**. This is a text file which will contain one or more **class** definitions. The extension to the filename should be **.java**. Note that this extension is four characters long. All the program code should reside in a class. By convention the name of the class should match the name of the file that holds the program. This means that the program illustrated above should be stored with a file name **First.java**. Also remember that Java is case sensitive. Therefore care has to be exercised while naming the file. To compile the above program execute the Java compiler **javac** at the command line as shown below:

```
C:> javac First.java
```

The Java compiler creates a file called **First.class**. This file contains the **bytecode** form of the program. This **bytecode** is interpreted by the interpreter. This means that the output of the java compiler is not a code which can be directly executed. Hence to actually run the program, you have to use the Java interpreter called **java**. You can do this by executing the command as shown below :

```
C:> java First
```

When the program runs you will get the following output :

```
My first program in Java
```

When you execute the Java interpreter as shown above, you are actually supplying the name of the class that you want the interpreter to execute. The interpreter automatically searches for the file by the specified name and which has a **.class** extension.

Now that you have seen how to successfully compile the program, let us begin our study of the Java programming language. The

student must note here that those sections where Java has similarities with languages like C and C++ shall be explained in brief and only an overview may be presented. Detailed programs may not be presented for every subtopic. Also wherever Java differs from these languages will be specifically indicated throughout the course material.

Let us study the line :

```
class First
```

We use the keyword **class** to declare that we are defining a new class. **First** is the name we have given to the class. Thus First is an **identifier**. The entire class definition, which includes all the members of that class is to be enclosed in a pair of curly braces. This is identical to the way they are used in C and C++.

The next line of code is :

```
public static void main(String args[])
```

This line begins the **main()** method. This is the line where program execution begins. At this point of time we shall not attempt to study the details of this line. It is enough to note that your program execution begins at this line and that all Java applications begin execution by calling **main()**. As we progress through our study we shall study this line in detail. It is also important to note that the Java compiler will compile classes that do not contain the **main()** method, however the Java interpreter cannot run these classes.

The brace bracket indicates the start of the body of the **main()** method. All the code of a particular method should be enclosed within a pair of braces.

The line -

```
System.out.println("My first program in Java");
```

will output

```
My first program in Java
```

on the screen. We use the **println()** method to output information on the console. In our example, **println()** outputs a string on the screen. **System** is a predefined class which provides access to the system and **out** is the output stream connected to the console. You should note that console input/output is mostly used for simple, utility programs. Modern computing environments are windowed and graphical in nature, therefore console input/output is not frequently used in real Java programs and applets. Also note that the **println()** statement ends with a semicolon (;). All statements in Java must end with a semicolon.

A Java program is a collection of comments, whitespace, identifiers, keywords, literals, separators, and operators. Now that we have studied a simple Java program, let us begin our study of these elements.

2.1 Check Your Progress.

1. Fill in the blanks.

- a) The extension to a Java source file should be _____ .
- b) The Java compiler creates a file which has the _____ extension, after compiling the source file.

2. Write True or False

- a) All program code in Java should reside in a class.
- b) A Java compiler compiles classes which do not contain a **main()** method.
- c) Output of the Java compiler is a code which can be directly executed.

2.2 COMMENTS, WHITESPACES, SEPARATORS

We have already studied that anything that has been entered as a comment is ignored by the compiler. Comments are useful to explain the operation of the program to anyone who is reading it. Java supports three types of comments.

2.2.1 Comments

Multiline comments : These begin with a **/*** and end with ***/**. Anything that is written between these two comments symbols is a comment and is ignored by the compiler. As the name suggests, these comments can be several lines long.

Single line comments : These begin with the symbol **//** and end at the end of line. Such types of comments do not have an ending symbol.

Documentation comments : These begin with **/**** and end with ***/**. This type of comment is used to produce an HTML file that documents the program. We shall not attempt to study this in detail.

2.2.2 White spaces

Java is a free form language which means that there are no specific indentation rules while writing programs. However, note that there has to be at least one whitespace character between each token, which has not been delineated by an operator or a separator. A whitespace can be a **space**, a **tab** or a **newline**.

2.2.3 Separators

Java uses the following separators :

- ;** to terminate a statement
- ,** to separate consecutive identifiers in declaring variables. Also used inside a **for** statement to chain statements
- .** to separate package names from subpackages and classes, to separate a variable or a method from a reference variable.
- ()** used to contain a list of parameters in a method definition as well as invocation. Also used for defining precedence in expressions, and surrounding cast types. Also used for containing expressions in control statements.
- { }** used to define blocks of code, for classes, methods

and local scopes, also used to contain values of automatically initialised arrays.

[] used to declare array types. Also used when dereferencing array values.

2.2 Check Your Progress.

1. Write True or False

- a) Java supports three types of comments.
- b) In Java, a space is not treated as a whitespace.
- c) , is a type of separator in Java.

2.3 IDENTIFIERS, LITERALS, AND KEYWORDS

2.3.1 Identifiers

Identifiers are names given to classes, methods, variables. An identifier can be a combination of uppercase and lowercase characters, numbers, the underscore and the dollar sign. But they cannot begin with a number. Also remember that Java is case sensitive so MAX is different from max.

BIG THREE_one count a2 \$first are all valid
identifiers

2a max-val first/last are not valid
identifiers

2.3.2 Literals

A literal is a **constant** value. Following are some examples of literals:

100 65.42 'A' "string literal"

As you can see the first is an **integer** literal, the second a **floating** point literal, the third is a **character** constant and the last is a **string**.

Integer Literals : Any whole number value is an integer literal. The bases that can be used for integer literals are decimal, octal and hexadecimal. Octal values in Java are denoted by a leading zero. Integer literals create an **int** type value which the Java compiler stores as a 32 bit value.

Floating point literals : These represent decimal values with a fractional component. They can be expressed either in the standard or in the scientific notation. By default floating literals are stored with double precision.

Boolean literals : boolean literals are only two logical values **true** or **false**. The values **true** and **false** do not convert to any numerical representation.

Character Literals : Character literals are 16 bit values which can be converted into integers and manipulated with the integer operators, like the addition and subtraction operators. A character literal is represented

inside a pair of single quotes eg. 'p'.

String Literals : These are specified by enclosing them in a pair of double quotes as shown in the example above. Remember that in Java string must begin and end on the same line, there is no line continuation escape sequence.

2.3.3 Keywords

Java has 48 keywords which when combined with the syntax of operators and separators form the definition of the Java language. Note that these keywords cannot be used as variable, method or class names. The keywords are :

abstract	const	finally	int	public	this
boolean	continue	float	interface	return	throw
break	default	for	long	short	throws
byte	do	goto	native	static	transient
case	double	if	new	strictfp	try
catch	else	implements	package	super	void
char	extends	import	private	switch	volatile
class	final	instanceof	protected	synchronized	while

The keywords **const** and **goto** have been defined but are not used. In addition, Java reserves : **true**, **false** and **null**. These are values defined by Java and these words cannot be used as names for variables, methods, classes etc.

The Java environment depends on a number of built in class libraries which contain numerous built in methods that provide support for input/output, string handling, networking etc. Thus Java in totality is a combination of the Java language itself, and its standard classes. The class libraries provide much of the functionality that comes with Java. So it is essential for the student to learn using the standard Java classes.

2.3 Check Your Progress.

1. Answer the following.

a) List the various types of literals supported by Java.

b) List the reserved words besides the keywords in Java.

2.4 DATA TYPES

Java defines the following simple data types :

Integers : This group includes **byte**, **short**, **int** and **long**. These are used for whole values which are signed numbers.

Floating point types : This group includes **float** and **double**. These represent values with fractional parts.

Characters : This group includes **char**, and they represent symbols in a character set. eg. letters, numbers.

Boolean : This is a special type for representing true/false values.

These simple data types form the basis of all the other types of data that you can create. It is important to note that these simple types represent single values and not complex objects. They are similar to the simple types found in most other non-object oriented languages. In Java all data types have a strictly defined range. This is essential to achieve the requirement of portability. This helps to make the programs architecture independant.

2.4.1 Integers

All the four integer data types viz. **byte**, **short**, **int** and **long** are **signed** i.e. positive and negative values. Java does not support unsigned integers. This differs from C/C++ where both signed and unsigned integers are supported.

byte : The smallest integer type is **byte**. This is a signed 8 bit type with a range from -128 to 127. A byte variable is declared using the keyword **byte**. eg.

```
byte b, b1;
```

Such types of variables are useful when working with a stream of data from a network or a file and also when working with raw binary data which may not be directly compatible with other built in types supported by Java.

short : **short** is signed 16-bit type which has a range of values form - 32,768 to 32,767. eg.

```
short s, s1;
```

int : This is the most commonly used integer data type with a range of values from -2,147,483,648 to 2,147,483,647. It is a signed 32 bit data type, Variables of type **int** are mainly used to control loops and to index arrays. It is important to note that anytime you have an integer expression which involves bytes, shorts and ints, and literal numbers, the entire expression will be promoted to the type **int** before the calculation is done. **int** is the most versatile and efficient data type among integers.

long : **long** is signed 64 bit and useful in situations where the **int** data type is not large enough to hold a particular value.

2.4.2 Floating point types

Floating point numbers are also known as real numbers. These are useful in expressions involving fractional values. There are two types of floating point types : **float** and **double**.

float : **float** represents single precision value. It uses 32 bits of storage. It can be useful when you need a fractional component but where the degree of precision required is not very high. A **float** variable is declared with the **float** keyword. The following example will illustrate :

```
float max, min;
```

double : Double precision is denoted by the keyword **double**. It uses 64 bits to store a value. All transcendental mathematical functions like `sin()`, `cos()`, `sqrt()` etc. return **double** values. Double precision is actually faster than single precision on some modern processors of today which have been optimised for high speed mathematical calculations.

2.4.3 Characters

The data type used to store characters is **char**. Remember however that **char** in Java is different than C/C++ where the **char** is an integer which occupies 8 bits. In Java, Unicode is used to represent **char**. Unicode defines a fully international character set that can represent all of the characters found in all human languages. Therefore the **char** data type requires 16 bits and the range of **char** is from 0 to 65,536. There are no negative character data types. Even though **char** is not treated as integers, you can add chars or increment the value of char variables.

2.4.4 Boolean

boolean is the simple data type for logical values. It has only one of the possible values : **true** or **false**. This is the type returned by all relational operators. **boolean** is also the type required by the conditional expressions in the control statements like **if** and **for**.

2.4 Check Your Progress.

1. Write True or False

- a) Integer data types are treated as objects in Java.
- b) boolean is a simple data type.
- c) In Java, the **char** data type is treated as integer.

2.5 VARIABLES

A variable is the basic unit of storage in a Java program. All variables must be declared before they are used. The general form of declaring a variable is :

```
type identifier [=value][,identifier[=value],...];
```


type is one of Java's atomic type or the name of a class or an interface and identifier is the name of the variable. Variable can be initialised at the time of declaration by giving an equal to sign and a value. We use a comma to separate variables if we are declaring more than one variable of the specified type. Following examples illustrate :

```
int a = 10, b, c;  
double radius;  
char a = 'a', ch;
```

Java also allows variables to be initialised dynamically by using any expression valid at the time the variable is declared.

The following example will illustrate :

```
class Circum  
{  
    public static void main(String args[])  
    {  
        float r = 4.2;  
        //initialise circumference dynamically  
        float cir = 2 * 3.14 * r  
        System.out.println("Circumference is "+ cir);  
    }  
}
```

In this example you can see that the variable r is initialised with a constant value. Variable cir is dynamically initialised to the circumference of the circle with radius r.

Scope and Lifetime of variables : Java allows variables to be declared in any block. A **block** defines the **scope**. This means that everytime you create a new block, you create a new scope. The scope determines what objects are visible to other parts of the program and it also determines the lifetime of the objects. In Java, the two major scopes are those defined by a class and those defined by a method. The class scope has several unique properties and attributes. These do not apply to the scope defined by a method. Let us for the moment see the scope defined within a method.

Variables declared inside a scope are not visible to code that is defined outside the scope. This means that when a variable is declared within a scope, it is localised to that scope and is protected from unauthorised access and modification. Scope rules provide the basis of encapsulation. Scopes can also be nested. The objects declared in the outer scope will be visible to the code within the inner scope, however the objects declared within the inner scope are not visible outside it.

In Java, variables are created when their scope is entered and destroyed when their scope is left. Once a variable goes out of its scope, it will no longer retain its value. This means that the lifetime of a variable is only within its scope. If a variable is initialised at the time it is declared, then that variable will be reinitialised everytime the block in which it is declared is entered.

2.6 TYPE CONVERSION AND CASTING

Automatic type conversion : When one type of data is assigned to another type of variable, an automatic conversion takes place if the following conditions are satisfied:

- the two types are compatible
- the destination type is larger than the source type.

In such a situation, a **widening conversion** takes place. eg. when a **byte** is converted to **int**, automatic conversion will take place. In automatic conversions, the numeric data types including integers and floats are compatible with each other. However, the numeric data types are not compatible with **char** or **boolean**. **char** and **boolean** are not compatible with each other either. Java also performs automatic type conversion when storing a literal integer constant into variables of type **byte**, **short** or **long**.

Casting Incompatible types : In a situation like where you want to convert an **int** data type to a **byte**, automatic conversion is not possible since **byte** is smaller than **int**. In such situations you have to use a cast. A **cast** is an explicit type conversion. It takes the following form :

(target-type) value

the *target-type* specifies the desired type to convert the specified value to.

```
eg. int a;  
byte b;  
b = (byte) a;
```

In this example, an **int** is cast to a **byte**. Here if the value of the integer is bigger than the range of a byte, then it will be reduced modulo byte's range.

When a floating point value is assigned to an integer, a different type of conversion i.e. truncation will occur and the fractional component will be lost. Also, if the size of the whole number is large to fit within the target integer type, then the value will be reduced modulo the target type's range.

Automatic Type promotion : In Java, all **bytes** and **shorts** are automatically promoted to **ints**. Also, if one operand is **long**, then the

whole expression is promoted to **long**. If one operand is a **float** operand, the entire expression is promoted to **float** and if any of the operands is **double**, the result is **double**.

2.7 ARRAYS

An array is group of variables of the same type and which have a common name. Arrays can be of any type and can have one or more dimensions. Any specific value or element in an array is accessed by its index. Remember that arrays in Java differ from the way they work in C/C++.

One dimensional array : A one dimensional array is a list of like typed variables. The general form for creating a one dimensional array is :

```
type var-name[ ] ;
```

here *type* is the base type of the array which determines the data type of the elements of the array. *var-name* is an array variable. You can declare an array as shown below :

```
int arr1[ ];
```

here arr1 is an array type variable, but it is important to note that with this declaration no array actually exists. The value of arr1 is set to **null**, which represents an array with no value. To link arr1 to a list of variables of type **int**, you must allocate one using the **new** operator. **new** is a special operator that allocates memory. The general form of **new** when applied to a one dimensional array is :

```
array-var = new type[size];
```

type specifies the type of data, *size* specifies the number of elements in the array. *array-var* is the variable that is linked to the array. The elements in the array allocated by **new** will automatically be initialised to zero. Thus in the above example if we wish to allocate ten elements to arr1 then we can do so as :

```
arr1 = new int[10];
```

arr1 now refers to an array of 10 integers and the elements initialised to zero.

You can combine array declaration and allocation in one step as :

```
int arr1 = new int[10];
```

An alternative form of declaring arrays is given below :

```
type [ ] var-name;
```

```
eg. int [ ] a1 = new int[5];
```

Here the square brackets come first after the type specifier and then the name of the array variable. This form is included mainly as a part of convenience.

All arrays are dynamically allocated in Java. Once you have allocated an array, you can access the elements of the array by specifying an index

in a square bracket. All array indexes start at zero. i.e. `arr1[0]` is the first element of `arr1`, `arr1[1]` is the second element and so on.

Arrays can be initialised at the time of their declaration. An array initialiser is a list of comma separated expressions surrounded by curly braces. The commas separate the values of the array elements. Java will automatically create an array large enough to hold the number of values specified in the array initialiser. There is no need of using **new** when initialising arrays at the time of declaration.

eg. `int odd_nos = {1, 3, 5, 7, 9, 11};`

Java run time system will check to be sure that all array indexes are in the correct range, it makes sure that you do not accidentally try to store or reference values outside the range of the array. This is different than in C/C++ where there is no bounds checking.

Multidimensional arrays :

Multidimensional arrays are arrays of arrays. In order to declare a multidimensional array variable you should specify each additional index using another set of square brackets. eg. if you wish to initialise a two dimensional array you can do so as follows :

`int array1[][] = new int[4][5];`

About pointers : Java does not support or allow pointers. This is because if it allows the use of pointers, Java applets would be allowed to breach the firewall between the Java execution environment and the host computer. Java is designed in such a way that as long as you stay within the confines of the execution environment, you will never need to use a pointer nor would there be any benefit in using one.

2.5 to 2.7 Check Your Progress.

1. Write True or False

- a) Objects declared in the outer scope will be visible to the code within the inner scope.
- b) Variables can be dynamically initialised in Java.
- c) In Java, bytes and shorts cannot be automatically promoted to `int`.
- d) When a floating point value is assigned to an integer, truncation occurs.
- e) All arrays are dynamically allocated in Java.

2.8 OPERATORS

Most of the operators in Java can be divided into the following groups : arithmetic, bitwise, relational and logical. Java also supports certain additional operators to handle special situations. Most operators in Java work in the same way as they do in C/C++. Let us now study these operators in detail.

2.8.1 Arithmetic Operators

These are useful in mathematical operations. The operands of arithmetic operators must be of numeric type. Arithmetic operators cannot be used on **boolean** types, but they can be used on **char** types. The following is the list of operators supported by Java :

Remember that the unary - operator negates the operand. When the division operator is applied to integer types, the fractional component of the result will be truncated. Also the modulus operator returns the remainder of the division operation. It can be applied to both integer and floating point types. However, in C it can be applied on to integer data types.

Operator	Result	Operator	Result
+	Addition	-	Subtraction (also unary -)
*	Multiplication	/	Division
++	Increment	--	Decrement
+=	Addition assignment	-=	Subtraction assignment
*=	Multiplication assignment	/=	Division assignment
%	Modulus	%=	Modulus assignment

The **arithmetic assignment operator** is used as follows :

eg. `a = a + 5;` can be written as

`a += 5;`

Here we have made use of the addition assignment operator. Both the statements perform the same operation. As we can see from the above table, there are assignment operators for all the arithmetic binary operators. The general form can therefore be expressed as :

`var op= expression;`

The assignment operators are shorthand for their equivalent long forms. They are also implemented more efficiently by the Java run-time system than their equivalent long forms.

Increment and Decrement operators : As in C/C++ the increment operator increases the value of the operand by 1, and the decrement operator decreases the value of the operand by 1. Both are unary operators and can appear in both the postfix and prefix forms.

eg. `a = a + 1;`

can be rewritten using the increment operator as :

`a++;`

or

++a;

Similarly with the decrement operator.

2.8.2 Bitwise Operators

Bitwise operators can be applied to the integer data types, **byte**, **char**, **short**, **int** and **long**. They act upon individual bits of the operands. As we have already studied all the integer types except **char** are signed integers. This means they can represent positive and negative values. Java uses the **2s complement** encoding method. This means that the negative numbers are represented by inverting all the bits in the value and then adding 1 to the result. Remember that the high-order bit determines the sign of an integer.

Bitwise Logical operators : The bitwise logical operators are & (AND) , | (OR) , ^ (XOR) and ~ (NOT). The truth table of the bitwise logical operators is given below :

Bitwise NOT (~) is also called as the bitwise complement operator and it inverts all of the bits of its operand.

A	B	A&B	A B	A^B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Bitwise AND (&) produces a 1 bit if both the operands are also 1 and a 0 bit otherwise.

Bitwise OR (|) : In bitwise OR if either of the bits in the operands is a 1, then the resultant bit is 1.

Bitwise XOR (^) : This operator combines the bits such that if exactly one operand is 1, then the result is a 1, otherwise the result is a 0. In bitwise XOR, whenever the second operand has a 0 bit, the first operand is unchanged.

The Left Shift : The left shift operator (<<) shifts all of the bits in a value to the left a specified number of times. The general form of the left shift operator is :

value << num

where *num* specifies the number of positions to left shift the value of *value*. For each shift left, the high order bit is shifted out and is lost, and a zero is brought in on the right. Shifting left by a value of 1, has the same effect as doubling the original value, therefore this can be used as an alternative way to multiplication by 2.

The Right Shift : This operator shifts all of the bits in a value to the right

the specified number of times. The general form of right shift is :

```
value >> num
```

Where *num* specifies the number of positions to right shift the value in *value*. When a value has bits that are shifted off, those bits are lost. Each time a value is shifted to the right, it gets divided by two and discards any remainder. This means that right shift can be used for integer division by 2. When shifting to the right, the leftmost bits exposed by the right shift are filled in with the previous contents of the top bit. This is called as sign extension and it preserves the sign of the negative numbers when you right shift.

The Unsigned Right Shift : In the right shift operator, the sign of the numbers is preserved. However in certain situations, especially when you are not shifting numeric values you may not want the sign extension to take place. You would generally want to shift a zero in the high order bit irrespective of what its initial value is. For this purpose you make use of the unsigned right shift operator `>>>` which always shifts zeroes into the high order bit. This is known as unsigned shift.

Bitwise Operator Assignments : All the bitwise operators have a shorthand form which is similar to that of algebraic operators. It combines the assignment with the bitwise operation eg.

```
a = a << 2; can also be written as
```

```
a <<= 2;
```

```
x = x & y; can also be written as
```

```
x &=y;
```

2.8.3 Relational Operators

Relational operators determine the relationship that one operand has to the other operand. The relational operators in Java are :

The result of these operations yields a **boolean** value. Any type in Java, including integers, floats, chars, and booleans can be compared for equality and inequality. Only integer, floating point and character operands may be compared to see which is less than or greater than the other.

Operator	Result	Operator	Result
<code>==</code>	Equal to	<code>!=</code>	Not equal to
<code>></code>	Greater than	<code>>=</code>	Greater than or equal to
<code><</code>	Less than	<code><=</code>	Less than or equal to

Java does not define **true** or **false** in the same way as C/C++. In C/C++, **true** implies any non zero value and **false** is a zero value. However, in Java, **true** and **false** are non numeric values. Therefore to test for zero or

non zero you must explicitly make use of one or more relational operators. The following example will illustrate :

In C/C++ you can write a block of code as :

```
int check;  
if(check) ....  
if(!check) ...
```

However, in Java you would be required to write as follows :

```
if(check == 0) ....  
if(check != 0) ...
```

This means in Java to test for zero or non zero you have to explicitly make use of one or more of the relational operators.

2.8.4 Boolean Logical Operators

These operators operate only on boolean operands. All the binary logical operators combine two boolean values to form a resultant boolean value.

Operator	Result	Operator	Result
&	Logical AND		Logical OR
^	Logical XOR		Short circuit OR
&&	Short circuit AND	!	Logical unary NOT
&=	AND assignment	=	OR assignment
^=	XOR assignment	==	Equal to
!=	Not equal to	?:	Ternary if-then-else

The logical Boolean operators &, | and ^ operate on boolean values in the same way that they operate on bits of an integer. The logical ! operator inverts the Boolean state, i.e. !true = false and !false = true.

Short Circuit Logical Operators : Short circuit operators are secondary versions of Boolean AND and OR operators. They are not found in most other computer languages. We can see from the truth table of the operators that the OR operator results in **true** when A is **true**, irrespective of whatever B is. Similarly the AND operator results in a **false** when A is **false**, irrespective of what B is. So if you use the short circuit OR (||) and short circuit AND (&&) forms rather than the | and & forms, Java does not evaluate the right hand operand because the result can be determined by the left hand operand alone.

2.8.5 The Assignment Operator

The assignment operator is the = sign. The general form of the assignment operator is :

```
var = expression;
```


Note that the type of *var* must be compatible with *expression*. Assignment operator allows you to create a chain of assignments which is an easy way to set a group of variables to a common value. eg.

```
int a, b, c;
a = b = c = 10;  is valid in Java.
```

2.8.6 The ?: operator

This operator works in the same way as it does in C/C++. The general form of this operator is :

exp1? exp2 :: exp3

exp1 is any expression that evaluates to a boolean value. If *exp1* is *true* then *exp2* is evaluated else *exp3* is evaluated. The result of the operation is that of the expression that is evaluated (either *exp2* or *exp3*). Note that both *exp2* and *exp3* should return the same type and the return type cannot be **void**.

2.8.7 Operator Precedence

The following table shows the order of precedence of the Java operators from highest to lowest. Remember that the square brackets are

Highest			
()	[]		
++	--	~	!
*	/	%	
+	-		
>>	>>>	<<	
>	>=	<	<=
==	!=		
&			
^			
&&			
?:			
=	op=		
Lowest			

2.8 Check Your Progress.

1. Fill in the blanks.

- Bitwise OR is a _____ operator.
- The _____ operator can be used as an alternative way to multiplication by 2.
- Boolean operators operate only on _____ operands.
- & has a _____ priority as compared to &&.

used for array indexing. The dot operator is used for dereferencing objects. Parenthesis are used to alter the precedence of an operation. Remember that using parenthesis do not negatively affect the performance of your program.

2.9 SUMMARY

We learn some java keywords, identifiers, literals. Literals are integer, floating point, Boolean, character and string.

Different datatypes used in java are int, byte, short, long, float, double, Boolean, char and string. Declaration of variables is

```
type identifier[=value][,identifier[=value]..]
```

Array: Array is group of variables of the same type and which have common name. Arrays can be any type and can have one or more dimensions.

```
type var-name[] = new type[size];
```

Java does not support or allow pointers.

Most of the operators in java can be divided into the following groups : arithmetic, bitwise, relational and logical.

Source : www.java-samples.com(Link)

2.10 CHECK YOUR PROGRESS - ANSWERS

2.1

1. a) .java
 b) .class
2. a) True
 b) True
 c) False

2.2

1. a) True
 b) False
 c) True

2.3

1. a) Integer literals, floating point literals, boolean literals, character literals and string literals.
 b) true, false and null.

2.4

1. a) False
 b) True
 c) False

2.5

1. a) True
b) True
c) False
d) True
e) True

2.6

1. a) logical
b) left shift
c) boolean
d) higher

2.11 QUESTIONS FOR SELF - STUDY

1. Explain with example how you will compile and run a Java program.

2. Write brief notes on :

- a) Comments
- b) White spaces
- c) Identifiers
- d) The unsigned right shift operator
- e) Short circuit operators in Java
- f) Character Data Types

3. Write short notes on :

- a) Separators in Java
 - b) Integer Data Types
 - c) Float Data Types
 - d) Scope and Life of variables.
 - e) Bitwise Logical operators.
4. What do you mean by type conversion? What is widening conversion? Explain automatic type promotion.
5. What is an array in Java? Describe how arrays can be initialised in Java.

2.12 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

() () ()

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.